

TradeRoute — Complete Product Specification Document

A Journey-Agnostic Multimodal Tradeoff Engine

Version: 1.0
Prepared for: 4-Hour Hackathon Build
Team Size: 4 Developers
Scope: Single Hero Flow (MVP) + Full Architecture Blueprint

TABLE OF CONTENTS

- 1. Executive Summary
 - 2. Brand Identity & Positioning
 - 3. Product Architecture & Information Hierarchy
 - 4. User Workflows & System Flows
 - 5. Technology Architecture
 - 6. Database & Data Models
 - 7. API Specifications
 - 8. UI/UX Design System
 - 9. 4-Hour Hackathon Build Plan
 - 10. Deployment & DevOps
 - 11. Security & Compliance
 - 12. Future Roadmap
-

1. EXECUTIVE SUMMARY

1.1 Product Definition

TradeRoute is a **decision-neutrality platform** for multimodal journey comparison. It does not sell tickets. It does not push any single mode. It answers one question with radical transparency: *“For this specific trip, what is the true cost, true time, and true convenience of every available option?”*

1.2 Core Value Proposition

- **For travelers:** Eliminate decision fatigue. See all modes (Own Vehicle, Bus, Train, Flight) as door-to-door totals on a single screen.
- **For the market:** No existing platform is mode-agnostic. Every incumbent (Google Maps, Make-MyTrip, IRCTC, RedBus) is incentivized to hide competing options. TradeRoute has no transaction commission—its only customer is the traveler.

1.3 Hero Flow (The 3-Minute Demo)

- 1. User enters Origin → Destination (e.g., Mumbai → Pune)
- 2. System computes 4 parallel journey scenarios with door-to-door normalization
- 3. Results displayed as comparison cards + tradeoff scatter plot
- 4. User adjusts “Value of Time” slider; recommendation updates in real time
- 5. User drills into any mode to see hidden cost breakdown and hour-by-hour timeline

1.4 Success Metrics (Hackathon)

- Demo completes in under 180 seconds
 - All 4 modes render with plausible data for 3 pre-loaded corridors
 - Slider interaction changes the recommendation badge
 - Zero booking/payment logic (scope exclusion)
-

2. BRAND IDENTITY & POSITIONING

2.1 Brand Name

TradeRoute

Rationale: “Trade” evokes the economic tradeoff at the heart of the product. “Route” anchors it in mobility. Together: the route you take is a trade you make with your time and money.

2.2 Tagline

“See the true cost of every journey.”

Alternative taglines for different contexts:

- Pitch: *“The decision layer travel has been missing.”*
- Consumer: *“Stop guessing. Start comparing.”*
- B2B: *“Mode-shift intelligence for modern mobility.”*

2.3 Brand Personality

Trait	Expression
Radically Transparent	Every cost is broken down. No hidden fees. No mode favoritism.
Intellectually Honest	Acknowledges uncertainty (“Parking cost estimated”). No false precision.
Empowering	Gives the user control via the time-value slider. The user decides, not the algorithm.
Clinically Clean	Information-dense but uncluttered. Data visualization over decoration.

2.4 Brand Voice

- **Direct:** “Driving costs 1,840. Here’s why.”
- **Neutral:** Never celebrates one mode over another. Never shames driving.
- **Educational:** “Depreciation adds 220 because every kilometer wears your vehicle.”
- **Actionable:** Every insight ends with a clear choice, not guilt.

2.5 Visual Identity

Color Palette

Token	Hex	Usage
Primary Navy	#0F172A	Headers, primary text, trust anchors
Accent Teal	#14B8A6	Primary actions, “Recommended” badge, efficient frontier line
Own Vehicle	#F59E0B (Amber)	Driving mode cards, scatter plot dots
Bus	#3B82F6 (Blue)	Bus mode cards, scatter plot dots
Train	#10B981 (Emerald)	Train mode cards, scatter plot dots
Flight	#8B5CF6 (Violet)	Flight mode cards, scatter plot dots
Surface	#F8FAFC	Page background
Card	FFFFFF	Content cards
Border	#E2E8F0	Dividers, card borders
Danger/Warning	#EF4444	High-cost alerts, delays
Success	#22C55E	Best tradeoff indicators

Typography

Role	Font	Weight	Size
Display / Logo	Inter	800	28px
H1 (Page Title)	Inter	700	32px
H2 (Section Title)	Inter	600	20px
Body	Inter	400	16px
Data / Numbers	JetBrains Mono	500	18-24px (tabular nums)
Caption / Meta	Inter	400	12px

Rationale: Inter is the standard for data-dense interfaces. JetBrains Mono ensures cost figures align vertically for easy scanning.

Logo Concept Abstract: Two diverging paths forming a subtle “T” lettermark, converging into a single decision point. The negative space between paths forms an upward arrow (optimization).
Literal (Hackathon-friendly): Text-only wordmark “TradeRoute” with a small teal chevron () replacing the crossbar of the “T”.
Favicon: Teal chevron inside a navy circle.

Iconography

- Use **Lucide React** icons (clean, consistent, lightweight)
- Mode icons: Car (own vehicle), Bus, TrainFront, Plane
- Cost icons: Fuel, Receipt, ParkingCircle, Clock, Wallet
- Action icons: ArrowRight, ChevronDown, SlidersHorizontal, MapPin

3. PRODUCT ARCHITECTURE & INFORMATION HIERARCHY

3.1 Site Map (Single-Page Application)



3.2 Page-by-Page Specification

3.2.1 Header

- **Height:** 64px, fixed top, backdrop-blur on scroll

- **Left:** Logo (chevron + “TradeRoute” in Inter 700, 20px)
- **Right:** “How it works” text link → opens methodology modal
- **Background:** White with 1px bottom border (#E2E8F0)

3.2.2 Hero / Input Section

- **Background:** Navy (#0F172A) with subtle gradient mesh (teal-to-navy, 5% opacity)
- **Headline:** “See the true cost of every journey” — White, Inter 800, 40px
- **Subheadline:** “Compare driving, bus, train, and flight as true door-to-door journeys. No hidden costs. No mode bias.” — Slate-300, Inter 400, 18px
- **Input Bar:** Centered card, white, rounded-2xl, shadow-lg, max-width 720px
 - **Origin:** Text input with MapPin icon, placeholder “Where from?”, autocomplete dropdown
 - **Swap Button:** Circular button between inputs, rotates 180° on click
 - **Destination:** Text input with MapPin icon, placeholder “Where to?”
 - **Vehicle Selector:** Dropdown (Hatchback / Sedan / SUV / “I don’t have a car”)
 - **Compare Button:** Full-width, teal bg, white text, ArrowRight icon, hover: darken 10%

3.2.3 Loading State

- **Animation:** 4 skeleton cards pulsing in staggered rhythm
- **Copy:** “Calculating door-to-door costs...” → “Fetching fuel rates...” → “Modeling depreciation...” → “Finding your best tradeoff...”
- **Duration:** Artificial 1.5s delay (even if data is instant) to build perceived value

3.2.4 Comparison Cards Grid

- **Layout:** 4-column grid on desktop, 2×2 on tablet, vertical stack on mobile
- **Card Anatomy (per mode):**
 - **Header band:** Mode color, 4px top border, mode icon + mode name
 - **Total Time:** Large number (JetBrains Mono, 28px) + “door-to-door” label
 - **Total Cost:** Large number (JetBrains Mono, 28px) + “all-in” label
 - **Badge row:** “Cheapest” / “Fastest” / “Best Tradeoff” (teal pill if recommended)
 - **Quick stats:** 3 mini-rows (Distance, Transfers/Stops, Comfort rating)
 - **CTA:** “View breakdown” secondary button (opens detail modal)

3.2.5 Tradeoff Scatter Plot

- **Container:** White card, rounded-xl, padding 24px
- **Title:** “Cost vs. Time Tradeoff”
- **X-axis:** Total Time (hours), 0 to max(journeys) + 1hr buffer
- **Y-axis:** Total Cost (), 0 to max(journeys) + 20% buffer
- **Data points:** 4 dots, colored by mode, 12px diameter
- **Efficient Frontier:** Dashed teal line connecting Pareto-optimal options (those not dominated by another option on both cost AND time)
- **Recommended dot:** Pulsing teal ring (2px stroke) around the dot matching user’s slider preference
- **Tooltip:** On hover, shows mode name, exact cost, exact time
- **Empty state message:** If only 1–2 modes available: “Only X modes serve this corridor. We’re working to add more.”

3.2.6 Value-of-Time Slider

- **Container:** White card, rounded-xl, margin-top 24px
- **Label:** “How much is your time worth?” + info tooltip
- **Slider:** Range input, styled with teal track, navy thumb
- **Range:** 0/hr to 2,000/hr (logarithmic scale feel: denser ticks at lower end)
- **Presets:**
 - “Budget Traveler” → 0/hr (optimize 100% for cost)
 - “Balanced” → 400/hr (Indian median wage proxy)
 - “Executive” → 1,500/hr (optimize 100% for time)
- **Dynamic update:** As slider moves, “Recommended” badge instantly recalculates and reassigns to the mode with lowest $(\text{Cost} + \text{Time} \times \text{Value})$
- **Formula display:** Small caption below slider: $\text{Total Journey Value} = ₹\text{Cost} + (\text{Hours} \times ₹\text{Value/hr})$

3.2.7 Detail Modal

- **Trigger:** Tap “View breakdown” on any comparison card
- **Structure:**
 - **Modal header:** Mode icon + Mode name + “×” close
 - **Timeline section:** Vertical timeline with 4–6 nodes
 - * Node types: Home → First Mile → Main Journey → Last Mile → Destination
 - * Each node: Time, activity, duration, cost (if any)
 - **Cost breakdown section:**
 - * Stacked horizontal bar chart showing component costs
 - * Itemized list below with icons: Fuel, Tolls, Parking, Depreciation, Ticket, First-mile, Last-mile, Airport time cost
 - * Each item: Label, amount, percentage of total, info tooltip explaining assumption
 - **Assumptions disclosure:** Collapsible section: “How we calculated this”
 - * Fuel price source (date)
 - * Depreciation model (/km)
 - * Parking assumption (city average)
 - * First/last mile assumption (auto/cab fare estimate)

3.2.8 Footer

- **Background:** Slate-50, border-top
- **Left:** “TradeRoute 2026 — Built for travelers, not transactions.”
- **Center:** “Data: OpenStreetMap, NHAI, IRCTC, RedBus, DGCA” (mock attribution)
- **Right:** Links: “Methodology” | “Privacy” | “GitHub” (open source badge)

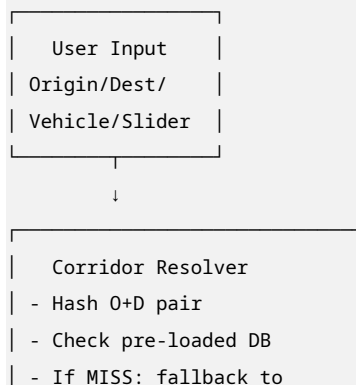
4. USER WORKFLOWS & SYSTEM FLOWS

4.1 Primary User Journey (Happy Path)

```
[User lands on TradeRoute]
↓
[Sees Hero + Input Bar]
↓
```

```
[Enters Origin: "Mumbai"]
  → [Geocoding API returns lat/lng + city context]
  → [Input shows "Mumbai, Maharashtra, India"]
  ↓
[Enters Destination: "Pune"]
  → [Geocoding API returns lat/lng + city context]
  → [Input shows "Pune, Maharashtra, India"]
  ↓
[Selects Vehicle: "Sedan" (default)]
  ↓
[Clicks "Compare"]
  → [Frontend validates both inputs have lat/lng]
  → [Frontend checks pre-loaded corridor cache]
  → [Cache HIT: Mumbai-Pune exists]
  → [Frontend renders Loading State (1.5s artificial)]
  → [Frontend renders Results Section]
    → Comparison Cards populated from cache
    → Scatter plot rendered with D3/Chart.js
    → Slider set to "Balanced" (₹400/hr) default
    → Recommendation badge calculated
  ↓
[User sees 4 cards + scatter plot]
  → [User drags slider to ₹800/hr]
  → [Frontend recalculates recommendation in <100ms]
  → [Badge moves from Bus to Train]
  → [Scatter plot "recommended" ring animates to Train dot]
  ↓
[User taps "Train" card → "View breakdown"]
  → [Detail Modal opens]
  → [Timeline: Home 8:00 AM → Cab to Dadar 8:30 AM → Train 9:00 AM → Arrive Pune 12:15 PM → Auto to
    ↪ destination 12:45 PM]
  → [Cost breakdown: Ticket ₹320 + Cab ₹180 + Auto ₹120 + Time value ₹300 = ₹920 total]
  ↓
[User closes modal]
  → [Back to results view]
  ↓
[User can modify search or share results]
```

4.2 System Flow (Data & Logic)





4.3 Edge Cases & Fallbacks

Edge Case	System Behavior
Origin = Destination	Show error toast: “Origin and destination cannot be the same.” Disable Compare button.
No driving route found	OSRM returns no route → Hide “Own Vehicle” card, show 3-mode comparison.

Table 4 – continued

Edge Case	System Behavior
No public transport data for corridor	Show “Own Vehicle” only + informational banner: “We’ re adding bus/train data for this route.”
Geocoding fails	Retry once → if still failing, show “Could not find that location. Try a city name.”
Slider at extreme values	At 0/hr: badge always goes to cheapest. At 2,000/hr: badge always goes to fastest. Show tooltip explaining this.
User selects “I don’ t have a car”	Hide “Own Vehicle” card entirely. Adjust scatter plot Y-axis range.
Mobile viewport	Cards stack vertically. Scatter plot becomes a horizontal bar chart (cost) with time labels.

5. TECHNOLOGY ARCHITECTURE

5.1 Architecture Pattern

Frontend-Heavy SPA with Static Data Layer

- No backend server required for MVP (hackathon scope)
- All journey data is pre-loaded as static JSON
- All calculations run client-side in JavaScript
- Geocoding calls external free API (Nominatim)
- Driving distance/time calls OSRM demo server or uses pre-computed distances

Post-hackathon evolution: Add a lightweight Node.js/Express API for dynamic data fetching, user preferences, and crowdsourced cost updates.

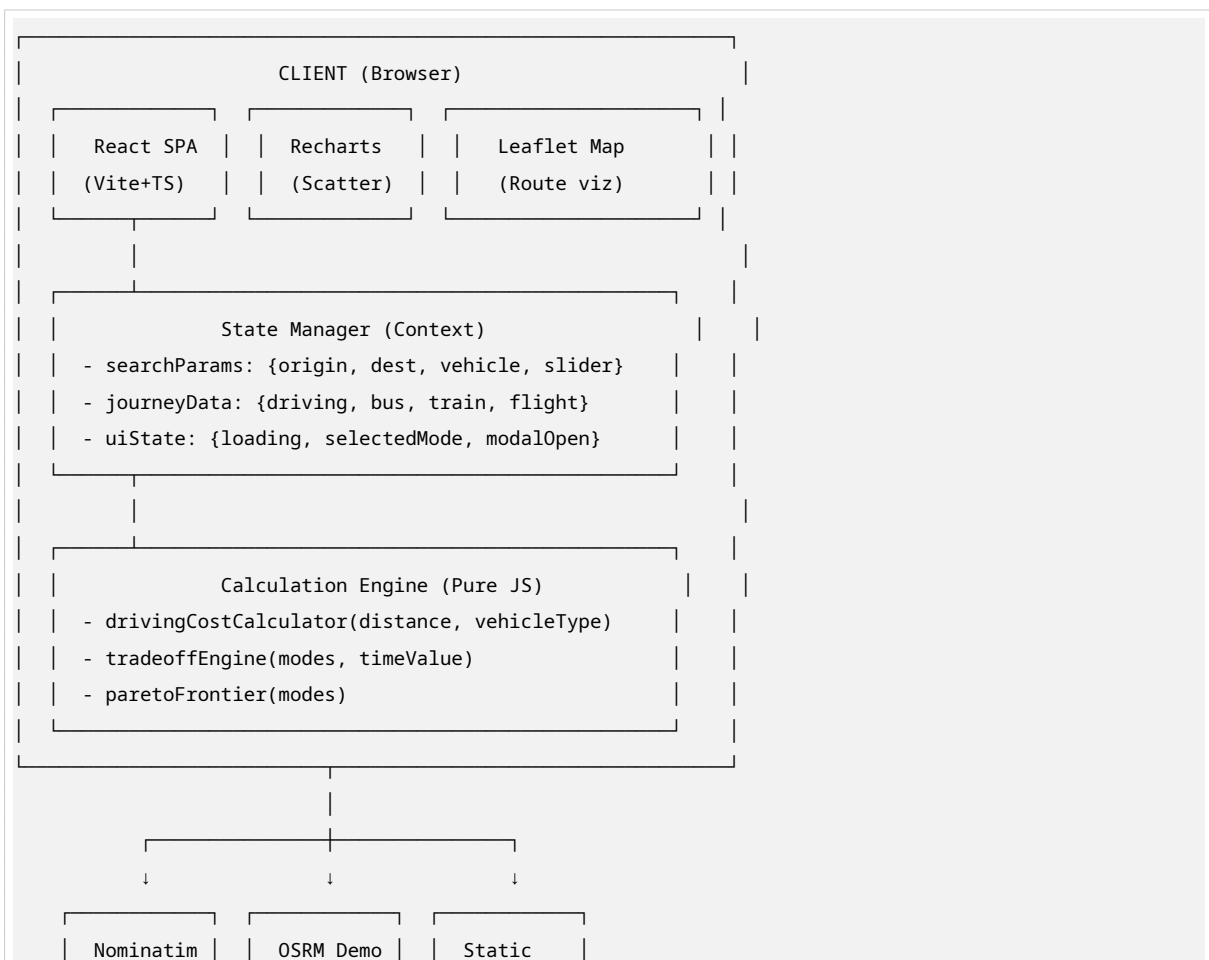
5.2 Tech Stack

Layer	Technology	Justification
Frontend Framework	React 18 (Vite)	Fast dev server, hot reload, component-based UI, excellent ecosystem
Language	TypeScript	Type safety for data models, prevents runtime errors in demo
Styling	Tailwind CSS	Rapid UI development, design system consistency, no CSS file management
UI Components	shadcn/ui + Radix	Accessible, composable primitives (Modal, Slider, Card, Tooltip)
Icons	Lucide React	Lightweight, consistent, tree-shakeable

Table 5 – continued

Layer	Technology	Justification
Charts	Recharts (React wrapper)	Declarative, responsive, easy scatter plot + bar chart
Maps	Leaflet (react-leaflet)	Open source, no API key needed for OSM tiles, lightweight
Routing / Distance	OSRM (Open Source Routing Machine)	Free, open, accurate driving distances. Use public demo server OR pre-compute
Geocoding	Nominatim (OpenStreetMap)	Free, no API key, sufficient for city-level geocoding
State Management	React Context + useReducer	Sufficient for MVP complexity. No Redux needed.
Build Tool	Vite	Instant HMR, optimized production build
Package Manager	pnpm	Fast, disk-space efficient
Deployment	Vercel	One-click deploy from GitHub, CDN, HTTPS, custom domain ready

5.3 System Diagram



(Geocode)	(Driving)	JSON DB
Free API	Free API	(Bus/Train
		/Flight)

5.4 File Structure

```
trade-route/
├─ public/
│  └─ data/
│     └─ corridors/
│        ├── mumbai-pune.json
│        ├── delhi-jaipur.json
│        ├── bangalore-mysore.json
│        └─ chennai-pondicherry.json
│  └─ favicon.svg
├─ src/
│  └─ components/
│     ├── Header.tsx
│     ├── HeroSection.tsx
│     ├── SearchBar.tsx
│     ├── LoadingState.tsx
│     ├── ResultsSection.tsx
│     ├── ComparisonCard.tsx
│     ├── TradeoffChart.tsx
│     ├── ValueSlider.tsx
│     ├── DetailModal.tsx
│     ├── Timeline.tsx
│     ├── CostBreakdown.tsx
│     └─ Footer.tsx
│  └─ hooks/
│     ├── useGeocoding.ts
│     ├── useJourneyCalculation.ts
│     └─ useTradeoffEngine.ts
│  └─ lib/
│     ├── constants.ts      # Fuel rates, depreciation, city averages
│     ├── utils.ts          # Formatters, calculators
│     └─ types.ts           # TypeScript interfaces
│  └─ data/
│     └─ corridorDatabase.ts # In-memory cache + loader
│  └─ context/
│     └─ AppContext.tsx     # Global state provider
│  └─ App.tsx
│  └─ main.tsx
│  └─ index.css
├─ index.html
├─ package.json
├─ tsconfig.json
├─ vite.config.ts
└─ tailwind.config.js
```

6. DATABASE & DATA MODELS

6.1 Data Strategy

For the 4-hour hackathon: **Zero database. Static JSON files.**

- All corridor data is hardcoded in `public/data/corridors/`
- All city averages (fuel, parking, first-mile) are in `src/lib/constants.ts`
- Post-hackathon: migrate to SQLite/PostgreSQL with a simple REST API.

6.2 TypeScript Interfaces

```
// src/lib/types.ts

interface GeoPoint {
  lat: number;
  lng: number;
  displayName: string;
}

interface SearchParams {
  origin: GeoPoint | null;
  destination: GeoPoint | null;
  vehicleType: 'hatchback' | 'sedan' | 'suv' | 'none';
  timeValuePerHour: number; // in rupees
}

interface JourneyMode {
  id: 'own_vehicle' | 'bus' | 'train' | 'flight';
  name: string;
  icon: string;
  color: string;

  // Door-to-door totals
  totalTimeMinutes: number;
  totalCostRupees: number;

  // Components
  mainJourney: {
    mode: string;
    distanceKm: number;
    durationMinutes: number;
    costRupees: number;
    operator?: string; // e.g., "RedBus", "Shatabdi Express", "IndiGo"
  };

  firstMile: {
    mode: string; // "walk", "auto", "cab", "metro"
    durationMinutes: number;
  };
}
```

```

    costRupees: number;
    distanceKm: number;
};

lastMile: {
    mode: string;
    durationMinutes: number;
    costRupees: number;
    distanceKm: number;
};

bufferTimeMinutes: number; // check-in, security, waiting

// Driving-specific
fuelCost?: number;
tollCost?: number;
parkingCost?: number;
depreciationCost?: number;

// Meta
isAvailable: boolean;
comfortRating: 1 | 2 | 3 | 4 | 5; // 1=poor, 5=excellent
reliabilityRating: 1 | 2 | 3 | 4 | 5;
}

interface CorridorData {
    id: string; // "mumbai-pune"
    origin: GeoPoint;
    destination: GeoPoint;
    drivingDistanceKm: number;
    drivingDurationMinutes: number;
    modes: {
        own_vehicle?: JourneyMode;
        bus?: JourneyMode;
        train?: JourneyMode;
        flight?: JourneyMode;
    };
};

interface TradeoffResult {
    modes: JourneyMode[];
    recommendedId: string;
    cheapestId: string;
    fastestId: string;
    paretoOptimalIds: string[];
}

interface TimelineNode {
    time: string; // "08:00"
    label: string;
    description: string;
}

```

```
durationMinutes: number;  
costRupees: number;  
icon: string;  
}
```

6.3 Static JSON Example (Corridor)

```
{  
  "id": "mumbai-pune",  
  "origin": {  
    "lat": 19.0760,  
    "lng": 72.8777,  
    "displayName": "Mumbai, Maharashtra"  
  },  
  "destination": {  
    "lat": 18.5204,  
    "lng": 73.8567,  
    "displayName": "Pune, Maharashtra"  
  },  
  "drivingDistanceKm": 155,  
  "drivingDurationMinutes": 195,  
  "modes": {  
    "own_vehicle": {  
      "id": "own_vehicle",  
      "name": "Own Vehicle",  
      "icon": "Car",  
      "color": "#F59E0B",  
      "totalTimeMinutes": 225,  
      "totalCostRupees": 1840,  
      "mainJourney": {  
        "mode": "Driving",  
        "distanceKm": 155,  
        "durationMinutes": 195,  
        "costRupees": 0  
      },  
    },  
    "firstMile": {  
      "mode": "walk",  
      "durationMinutes": 10,  
      "costRupees": 0,  
      "distanceKm": 0.8  
    },  
    "lastMile": {  
      "mode": "walk",  
      "durationMinutes": 10,  
      "costRupees": 0,  
      "distanceKm": 0.8  
    },  
  },  
  "bufferTimeMinutes": 10,  
  "fuelCost": 775,  
  "tollCost": 520,  
  "parkingCost": 300,  
}
```

```

    "depreciationCost": 245,
    "isAvailable": true,
    "comfortRating": 4,
    "reliabilityRating": 3
  },
  "bus": {
    "id": "bus",
    "name": "Bus",
    "icon": "Bus",
    "color": "#3B82F6",
    "totalTimeMinutes": 270,
    "totalCostRupees": 650,
    "mainJourney": {
      "mode": "AC Sleeper Bus",
      "distanceKm": 155,
      "durationMinutes": 240,
      "costRupees": 450,
      "operator": "RedBus / MSRTC"
    },
    "firstMile": {
      "mode": "auto",
      "durationMinutes": 15,
      "costRupees": 80,
      "distanceKm": 3
    },
    "lastMile": {
      "mode": "auto",
      "durationMinutes": 15,
      "costRupees": 120,
      "distanceKm": 4
    },
    "bufferTimeMinutes": 0,
    "isAvailable": true,
    "comfortRating": 3,
    "reliabilityRating": 3
  },
  "train": {
    "id": "train",
    "name": "Train",
    "icon": "TrainFront",
    "color": "#10B981",
    "totalTimeMinutes": 225,
    "totalCostRupees": 920,
    "mainJourney": {
      "mode": "Shatabdi Express (Chair Car)",
      "distanceKm": 192,
      "durationMinutes": 180,
      "costRupees": 320,
      "operator": "Indian Railways"
    },
    "firstMile": {

```

```

    "mode": "cab",
    "durationMinutes": 20,
    "costRupees": 180,
    "distanceKm": 8
  },
  "lastMile": {
    "mode": "auto",
    "durationMinutes": 20,
    "costRupees": 120,
    "distanceKm": 5
  },
  "bufferTimeMinutes": 5,
  "isAvailable": true,
  "comfortRating": 4,
  "reliabilityRating": 5
},
"flight": {
  "id": "flight",
  "name": "Flight",
  "icon": "Plane",
  "color": "#8B5CF6",
  "totalTimeMinutes": 165,
  "totalCostRupees": 3400,
  "mainJourney": {
    "mode": "Commercial Flight",
    "distanceKm": 120,
    "durationMinutes": 55,
    "costRupees": 2400,
    "operator": "IndiGo / Air India"
  },
  "firstMile": {
    "mode": "cab",
    "durationMinutes": 45,
    "costRupees": 600,
    "distanceKm": 25
  },
  "lastMile": {
    "mode": "cab",
    "durationMinutes": 30,
    "costRupees": 400,
    "distanceKm": 18
  },
  "bufferTimeMinutes": 35,
  "isAvailable": true,
  "comfortRating": 4,
  "reliabilityRating": 4
}
}
}

```


6.4 Constants File

```
// src/lib/constants.ts

export const FUEL_RATES = {
  petrol: 106.5, // ₹/liter (Mumbai avg, update per city)
  diesel: 92.3,
  cng: 79.0
};

export const VEHICLE_EFFICIENCY = {
  hatchback: { kmPerLiter: 18, fuelType: 'petrol' },
  sedan: { kmPerLiter: 16, fuelType: 'petrol' },
  suv: { kmPerLiter: 12, fuelType: 'diesel' }
};

export const DEPRECIATION_RATES = {
  // ₹ per km (simplified model: insurance + maintenance + value loss)
  hatchback: 4.5,
  sedan: 5.5,
  suv: 7.0
};

export const CITY_AVERAGES = {
  'mumbai': { parkingPerDay: 300, firstMileAuto: 80, firstMileCab: 200 },
  'pune': { parkingPerDay: 150, firstMileAuto: 60, firstMileCab: 150 },
  'delhi': { parkingPerDay: 250, firstMileAuto: 70, firstMileCab: 180 },
  'jaipur': { parkingPerDay: 100, firstMileAuto: 50, firstMileCab: 120 },
  'bangalore': { parkingPerDay: 200, firstMileAuto: 75, firstMileCab: 190 },
  'mysore': { parkingPerDay: 80, firstMileAuto: 40, firstMileCab: 100 }
  // ... extend for all demo cities
};

export const TOLL_RATES: Record<string, number> = {
  'mumbai-pune': 520,
  'delhi-jaipur': 380,
  'bangalore-mysore': 280,
  'chennai-pondicherry': 320
};

export const SLIDER_PRESETS = [
  { label: 'Budget Traveler', value: 0 },
  { label: 'Balanced', value: 400 },
  { label: 'Executive', value: 1500 }
];
```

7. API SPECIFICATIONS

7.1 External APIs (Used in MVP)

7.1.1 Nominatim Geocoding

```
GET https://nominatim.openstreetmap.org/search?q={query}&format=json&limit=5&countrycodes=in
```

Headers: User-Agent: TradeRoute/1.0

Response:

```
[
  {
    "place_id": 123456,
    "display_name": "Mumbai, Maharashtra, India",
    "lat": "19.0760",
    "lon": "72.8777",
    "type": "city"
  }
]
```

Rate Limit: 1 request/second (generous for demo)

Fallback: Pre-loaded city list for quick-select buttons.

7.1.2 OSRM Driving Distance (Optional)

```
GET https://router.project-osrm.org/route/v1/driving/{lon1},{lat1};{lon2},{lat2}?overview=false
```

Response:

```
{
  "routes": [
    {
      "distance": 155000, // meters
      "duration": 11700  // seconds
    }
  ]
}
```

Hackathon Strategy: Pre-compute distances for demo corridors. Do NOT call OSRM during the live demo (network risk). Use pre-loaded `drivingDistanceKm` from JSON.

7.2 Internal “APIs” (Client-Side Functions)

7.2.1 Driving Cost Calculator

```
function calculateDrivingCost(
  distanceKm: number,
  vehicleType: VehicleType,
  corridorId: string
): DrivingCostBreakdown {
  const efficiency = VEHICLE_EFFICIENCY[vehicleType];
  const fuelNeeded = distanceKm / efficiency.kmPerLiter;
  const fuelCost = fuelNeeded * FUEL_RATES[efficiency.fuelType];
```

```

const tollCost = TOLL_RATES[corridorId] || Math.round(distanceKm * 2.5);
const depreciationCost = Math.round(distanceKm * DEPRECIATION_RATES[vehicleType]);

return {
  fuelCost: Math.round(fuelCost),
  tollCost,
  parkingCost: 0, // destination parking, added later
  depreciationCost,
  total: Math.round(fuelCost + tollCost + depreciationCost)
};
}

```

7.2.2 Tradeoff Engine

```

function calculateTradeoff(
  modes: JourneyMode[],
  timeValuePerHour: number
): TradeoffResult {
  const timeValuePerMinute = timeValuePerHour / 60;

  // Calculate total journey value for each mode
  const scoredModes = modes.map(mode => ({
    ...mode,
    journeyValue: mode.totalCostRupees + (mode.totalTimeMinutes * timeValuePerMinute)
  }));

  // Find cheapest, fastest, recommended
  const cheapest = scoredModes.reduce((min, m) => m.totalCostRupees < min.totalCostRupees ? m : min);
  const fastest = scoredModes.reduce((min, m) => m.totalTimeMinutes < min.totalTimeMinutes ? m : min);
  const recommended = scoredModes.reduce((min, m) => m.journeyValue < min.journeyValue ? m : min);

  // Pareto frontier: modes not dominated on both cost AND time
  const paretoOptimal = scoredModes.filter(mode =>
    !scoredModes.some(other =>
      other.id !== mode.id &&
      other.totalCostRupees <= mode.totalCostRupees &&
      other.totalTimeMinutes <= mode.totalTimeMinutes &&
      (other.totalCostRupees < mode.totalCostRupees || other.totalTimeMinutes < mode.totalTimeMinutes)
    )
  );

  return {
    modes: scoredModes,
    recommendedId: recommended.id,
    cheapestId: cheapest.id,
    fastestId: fastest.id,
    paretoOptimalIds: paretoOptimal.map(m => m.id)
  };
}

```

8. UI/UX DESIGN SYSTEM

8.1 Component Library (shadcn/ui base)

Component	shadcn Source	Customization
Button	npx shadcn add button	Teal primary, slate secondary, rounded-lg
Card	npx shadcn add card	1px border, subtle shadow, hover: shadow-md
Slider	npx shadcn add slider	Teal track, navy thumb, custom tick marks
Dialog	npx shadcn add dialog	Max-width 640px, rounded-xl, backdrop blur
Tooltip	npx shadcn add tooltip	Slate bg, white text, 8px padding
Badge	npx shadcn add badge	Teal variant for “Recommended”, amber for “Fastest”, emerald for “Cheapest”
Input	npx shadcn add input	48px height, slate border, focus: teal ring
Select	npx shadcn add select	Vehicle type dropdown
Skeleton	npx shadcn add skeleton	Loading state pulses
Toast	npx shadcn add toast	Error messages, copy confirmation

8.2 Responsive Breakpoints

Breakpoint	Width	Layout
Mobile	< 640px	Single column, stacked cards, horizontal bar chart instead of scatter
Tablet	640-1024px	2×2 card grid, scatter plot below
Desktop	> 1024px	4-column card grid, scatter plot + slider side-by-side

8.3 Animation & Motion

Interaction	Animation	Duration	Easing
Page load	Fade in + translateY(20px→0)	600ms	ease-out
Compare button click	Button scales 0.95, loading state fades in	200ms	ease-in-out
Cards appear	Staggered fade in (100ms delay each)	400ms	ease-out
Slider drag	Recommendation badge cross-fades	150ms	ease
Scatter plot dots	Scale from 0→1 on load	500ms	spring(1, 0.5)

Table 8 – continued

Interaction	Animation	Duration	Easing
Modal open	Backdrop fade, content scale 0.95→1	200ms	ease-out
Swap button	Rotate 180°	300ms	ease-in-out

Library: framer-motion for React animations (lightweight, declarative).

8.4 Accessibility (a11y)

- **Color contrast:** All text meets WCAG AA (4.5:1 minimum). Navy on white = 12.6:1.
- **Focus indicators:** 2px teal outline on all interactive elements.
- **Screen reader:** All mode cards are `article` regions with `aria-label`. Cost figures use `aria-describedby` by linking to breakdown.
- **Keyboard navigation:** Tab order: Input → Swap → Input → Vehicle → Compare → Cards → Slider → Modal triggers.
- **Reduced motion:** Respect `prefers-reduced-motion` media query (disable stagger animations).

9. 4-HOUR HACKATHON BUILD PLAN

9.1 Team Roles

Role	Developer	Responsibilities
Dev 1: UI Lead	Frontend specialist	Header, Hero, SearchBar, LoadingState, Footer, Tailwind config, animations
Dev 2: Data & Logic	TypeScript/algorithm strong	Types, constants, corridor JSONs, driving calculator, tradeoff engine, geocoding hook
Dev 3: Visualization	D3/Chart.js strong	ComparisonCard component, TradeoffChart (scatter plot), ValueSlider, badges
Dev 4: Detail & Polish	UX-focused	DetailModal, Timeline, CostBreakdown, responsive layout, demo script, pitch prep

9.2 Hour-by-Hour Timeline

Hour 0:00–0:30 — SETUP & ALIGNMENT

- All 4 devs: `git clone` repo template, `pnpm install`, verify dev server runs
- Dev 1: Initialize Tailwind, shadcn/ui, install Lucide, Framer Motion
- Dev 2: Create `types.ts`, `constants.ts`, first corridor JSON (Mumbai-Pune)
- Dev 3: Install Recharts, verify scatter plot renders with dummy data
- Dev 4: Write demo script draft, identify 3 corridors to pre-load
- **Checkpoint:** All devs confirm `localhost:5173` loads without errors.

Hour 0:30–1:30 — SKELETON & INPUT

- Dev 1: Build Header, HeroSection, SearchBar (inputs + swap button + vehicle select)
- Dev 2: Build useGeocoding hook (Nominatim integration), `corridorDatabase.ts` loader
- Dev 3: Build ComparisonCard component (static, with all 4 variants styled)
- Dev 4: Build LoadingState skeleton, Footer, methodology modal shell
- **Checkpoint:** User can type origin/destination, see autocomplete, click Compare → LoadingState appears.

Hour 1:30–2:30 — DATA PIPELINE & CARDS

- Dev 2: Wire useJourneyCalculation hook: reads corridor JSON, computes driving cost from constants, assembles 4 JourneyMode objects
- Dev 1: Wire SearchBar → AppContext → ResultsSection. ResultsSection renders 4 ComparisonCards with real data.
- Dev 3: Add badges (Cheapest, Fastest) to cards. Style “Recommended” badge.
- Dev 4: Build DetailModal shell with tabs (Timeline / Cost Breakdown)
- **Checkpoint:** Clicking Compare on “Mumbai → Pune” shows 4 cards with realistic numbers.

Hour 2:30–3:15 — CHARTS, SLIDER & RECOMMENDATION

- Dev 3: Build TradeoffChart (Recharts scatter plot). Plot 4 dots. Add efficient frontier line.
- Dev 3: Build ValueSlider with presets. Wire to useTradeoffEngine.
- Dev 2: Implement calculateTradeoff function. Wire slider change → recalculate recommendation → update badge + chart ring.
- Dev 4: Build Timeline component inside DetailModal. Build CostBreakdown stacked bar.
- **Checkpoint:** Slider drag changes recommendation badge. Chart dot pulses on recommended mode.

Hour 3:15–3:45 — DETAIL MODAL & RESPONSIVE

- Dev 4: Complete DetailModal with Timeline + CostBreakdown for all 4 modes.
- Dev 1: Add responsive breakpoints (mobile card stack, tablet 2×2).
- Dev 2: Add error handling (same city, no route, geocode fail).
- Dev 3: Add animations (card stagger, modal transition, slider smoothness).
- **Checkpoint:** Full happy path works on mobile and desktop. Detail modal opens for all modes.

Hour 3:45–4:00 — DEMO PREP & DEPLOY

- All: Run through demo script 2×. Identify and fix any jank.
- Dev 1: Deploy to Vercel (`vercel --prod`).
- Dev 4: Finalize pitch talking points. Prepare 3 corridor quick-select buttons as backup.
- Dev 2: Verify all JSON data is accurate and defensible.
- Dev 3: Screenshot key frames for pitch deck backup.
- **Checkpoint:** Live URL loads. Demo completes in <3 minutes. Pitch is rehearsed.

9.3 Pre-Hackathon Preparation (Do Before the Event)

Task	Owner	Time
Research and validate 3 corridor datasets (Mumbai-Pune, Delhi-Jaipur, Bangalore-Mysore)	Team	2 hours
Create repo template with Vite + React + TS + Tailwind + shadcn initialized	Dev 1	30 min
Pre-install all dependencies (Recharts, Framer Motion, Leaflet, Lucide)	Dev 1	15 min
Verify Nominatim geocoding works for demo city names	Dev 2	15 min
Write static JSON files for all 3 corridors with realistic data	Dev 2	1 hour
Create pitch deck template (5 slides: Problem, Solution, Demo, Market, Ask)	Dev 4	1 hour
Rehearse demo narrative 3×	Team	30 min

10. DEPLOYMENT & DEVOPS

10.1 Hackathon Deployment

Platform: Vercel

Steps: 1. Push code to GitHub repository

1. Push code to GitHub repository
2. Import repo into Vercel dashboard
3. Framework preset: Vite
4. Build command: `pnpm build`
5. Output directory: `dist`
6. Environment variables: None required for MVP
7. Deploy → receive `https://trade-route-hackathon.vercel.app`

Why Vercel:

- Zero-config for Vite/React
- Automatic HTTPS
- Edge CDN (fast global load)
- Instant rollback if demo breaks
- Free tier generous enough for hackathon traffic

10.2 Post-Hackathon Infrastructure Evolution

```
Phase 1 (MVP): Static SPA on Vercel
↓
Phase 2 (Dynamic): Add Node.js/Express API on Railway/Render
- PostgreSQL for corridor database
- Redis for geocoding cache
- Cron job to refresh fuel prices
↓
Phase 3 (Scale): Microservices on AWS/GCP
- Route calculation service (OSRM self-hosted)
- Price aggregation service (scrapers for bus/train/flight APIs)
- User preference service (auth, saved routes, time-value profiles)
- CDN: CloudFront / Cloudflare
```

11. SECURITY & COMPLIANCE

11.1 Hackathon Scope (No PII, No Payments)

- **No user accounts:** No authentication needed.
- **No personal data:** Origin/destination are transient; not stored server-side.
- **No financial transactions:** App ends at comparison. Booking links go to external platforms.
- **HTTPS:** Provided by Vercel automatically.
- **CSP:** Basic Content-Security-Policy via Vercel headers.

11.2 Future Compliance Roadmap

- **GDPR/CCPA:** If user accounts added, implement data export/deletion.
- **PCI-DSS:** Only relevant if in-app booking added (not recommended; affiliate links safer).
- **Accessibility:** WCAG 2.1 AA compliance audit before public launch.

12. FUTURE ROADMAP

12.1 Near-Term (0–3 Months)

- **Dynamic corridor addition:** Admin UI to add new O-D pairs without code deploy
- **Real-time fuel price API:** Integration with Indian Oil / PPAC daily price feeds
- **Toll calculator API:** Integration with NHAI / state toll authority APIs
- **Flight price scraper:** Basic Skyscanner/Amadeus affiliate API for real flight costs
- **Train seat availability:** IRCTC API integration (if available)
- **Bus price scraper:** RedBus API / ZingBus API
- **Carpool mode:** Add BlaBlaCar-style ride-sharing as 5th comparison option

12.2 Mid-Term (3–12 Months)

- **User accounts:** Save frequent routes, set default time-value preference
- **Carbon footprint layer:** Add CO₂ kg per mode to comparison (emissions data from DEFRA/IPCC)

- **Reliability scoring:** Historical delay data for trains/buses (crowdsourced + official)
- **Mobile app:** React Native or Flutter wrapper for native experience
- **B2B API:** Allow employers to embed TradeRoute in their commute benefit portals
- **City-level analytics:** Aggregate anonymized search data to show city planners where mode-shift potential exists

12.3 Long-Term (1–3 Years)

- **Global expansion:** Localized fuel prices, tolls, and public transport data for 50+ countries
- **Predictive pricing:** ML model forecasting bus/flight price trends (“Book now or wait?”)
- **Subscription model:** “TradeRoute Pro” for frequent travelers with saved routes, price alerts, and carbon tracking
- **Affiliate monetization:** Neutral booking links (same commission regardless of mode) to preserve decision-neutrality
- **Policy toolkit:** Open-data dashboards for transit agencies showing where their service is uncompetitive on cost/time

APPENDIX A: DEMO SCRIPT (3 Minutes)

[0:00–0:20] The Hook

“Last month, I drove from Mumbai to Pune. I thought it cost me 800 in petrol. It actually cost me 1,840 when you count tolls, parking, and the fact that my car lost value with every kilometer. But I never knew that—because no app shows the true cost. Until now.”

[0:20–0:45] The Problem

“Today, if you want to compare driving, bus, train, and flight, you need four apps, four tabs, and a calculator. Google Maps tells you driving takes 3 hours—but hides the 520 toll. MakeMyTrip shows a 2,400 flight—but hides the 1,000 cab ride to the airport. Every app is lying by omission because they make money when you choose THEIR mode.”

[0:45–1:30] The Demo — Input & Results

Clicks quick-select: Mumbai → Pune

“Enter your route. In 2 seconds, TradeRoute shows all four modes as true door-to-door journeys. Driving: 3 hours 45 minutes, 1,840 all-in. Bus: 4.5 hours, 650. Train: 3.75 hours, 920. Flight: 2.75 hours door-to-door, 3,400.”

[1:30–2:15] The Demo — Tradeoff & Slider

Points to scatter plot

“But here’s the magic. This scatter plot shows cost versus time. The teal line is the efficient frontier—modes that aren’t dominated by another option. Now, everyone values time differently. A student values money. A CEO values time. So we built this slider.”

Drags slider from 0 to 800/hour

“At zero—pure budget—the bus wins. At 400 per hour—India’s median wage—the train becomes optimal. At 1,500—the train still wins because the flight’s airport time kills it. TradeRoute doesn’t decide for you. It just shows you what YOUR time is worth.”

[2:15–2:45] The Demo — Detail Drill

Taps Train card → View Breakdown

“Drill into any mode and see exactly where the money and time go. The train ticket is only 320. But you need a cab to Dadar, an auto in Pune, and 20 minutes of buffer. That’s the truth. No app has ever shown this.”

[2:45–3:00] The Close

“TradeRoute is not a booking app. It’s a decision layer. We don’t make money when you book—we make money when you trust us to tell the truth. In a world of hidden costs and mode bias, radical transparency is the product. Thank you.”

APPENDIX B: MOCK DATA SOURCES & VALIDATION

Corridor	Driving Dist.	Driving Time	Bus Fare	Train Fare	Flight Fare	Sources
Mumbai–Pune	155 km	3h 15m	450	320 (Shatabdi CC)	2,400	Google Maps RedBus, IRCTC, Skyscanner
Delhi–Jaipur	280 km	5h 00m	550	450 (Rajdhani)	2,200	Google Maps RedBus, IRCTC, Skyscanner
Bangalore–Mysore	145 km	3h 00m	350	180 (Shatabdi)	3,800	Google Maps RedBus, IRCTC, Skyscanner

Note: Prices are approximate averages for demo purposes. In production, integrate live APIs.

APPENDIX C: COMPETITIVE POSITIONING MATRIX

Competitor	Their Angle	TradeRoute Differentiator
Google Maps	“Get there fastest”	Shows TRUE cost, not just time. Mode-agnostic.
MakeMyTrip / Cleartrip	“Book cheapest flight”	No booking bias. Shows flight vs. train honestly.
IRCTC / RedBus	“Book our mode”	Compares against driving and flying.
Rome2Rio	“How to get anywhere”	Focuses on cost accuracy and personalization, not just connectivity.
Hyperlocal apps (Citymapper)	“Urban transit only”	Covers intercity + intracity. Includes own vehicle.

END OF DOCUMENT

Prepared by: Senior Panel — Multidisciplinary Intelligence Unit

For: 4-Person Hackathon Team

Theme: Smart Cities & Mobility

Date: July 2026